SMART ROUTE FINDER

Shortest Path Analysis using Dijkstra's Algorithm

Discrete Mathematics and Graph Theory (MAT2002)

PROJECT & VIVA PREPARATION GUIDE

Team Member Name	Registration ID
Yash Janiyani	24BCE11394
Himanshu Vishwakarma	25MIM10119
Prashant Raj	24BCE11371
Abhishek	24BCE11402
Mradul Agrawal	24BCE11403
Devraj Srivastava	24BCE10050

Course: Discrete Mathematics and Graph Theory (MAT2002)
School: School of Computer Science & Engineering (SCOPE)
Topic: Shortest Path Algorithm for Directed Graphs
Implementation: React + Vite + Tailwind CSS + Manual Dijkstra JS

1. Project Overview

Smart Route Finder is an interactive web-based educational project built for the college subject **Discrete Mathematics and Graph Theory (MAT2002)**. The project demonstrates how directed graphs with non-negative edge weights can be represented computationally using adjacency matrices and lists, and how **Dijkstra's Algorithm** efficiently finds the shortest path between any source and destination vertex.

Problem Statement: Given a directed graph $G = (V, E)$ with positive edge weights $w(u, v) > 0$, a specified source vertex s, and a target destination d, calculate the minimum cumulative traversal distance and reconstruct the exact sequence of vertices forming the optimal route.

Key Deliverables of the Web App:

• Interactive Graph Construction: Add vertices and directed weighted edges with full validation.

• Visual Canvas: Draggable SVG nodes, directed arrows with arrowheads, and edge weight tags.

• Shortest Path Calculation: Manual Dijkstra implementation executing distance relaxation.

• Dynamic Adjacency Matrix: Automatically rendered $V \times V$ matrix matching MAT2002 syllabus.

• Step-by-Step Trace Table: Transparent execution trace showing vertex selection and relaxation.

2. MAT2002 Syllabus Connection

This project directly maps to the **Graphs and Matrix Representation** module of the MAT2002 course syllabus:

Syllabus Topic	Project Implementation Feature
Directed Graphs (Digraphs)	Edge direction $u \rightarrow v$ enforced with directed arrows
Weighted Graphs	Non-negative numeric weights $w(u, v) > 0$ assigned to edges
Adjacency Matrix	Dynamic $V \times V$ table displaying 0, edge weights, and ∞
Shortest Path Algorithm	Step-by-step manual implementation of Dijkstra's algorithm

3. How the Application Works

1. **Graph Creation:** The user initializes a graph either by loading the default MAT2002 demo graph (Vertices: A, B, C, D, E) or creating a custom graph via *Section A (Add Vertex)* and *Section B (Add Directed Edge)*.
2. **Representation:** Vertices are stored in React state array vertices, and directed edges are stored as { from, to, weight } objects.
3. **Visualization:** The graph is rendered on an SVG canvas. Vertices are positioned circularly and can be dragged.

Directed edges draw curved arrows with weight badges.
4. **Path Selection:** Source and Destination vertices are chosen from dropdown selectors.
5. **Algorithm Execution:** Clicking **'Find Shortest Path'** triggers runDijkstra(), which computes minimum distances, logs relaxation steps, and reconstructs the optimal path.
6. **Output & Trace:** The shortest path is highlighted on the canvas in green, the exact path sequence and total distance are displayed in a result card, the step-by-step relaxation trace is populated, and the adjacency matrix updates dynamically.

4. Dijkstra's Algorithm & Relaxation

Dijkstra's Algorithm is a greedy single-source shortest-path algorithm for graphs with non-negative edge weights. It maintains three essential structures during execution:

• **Distance Array (dist[]):** Stores the current minimum tentative distance from source *s* to each vertex *v*. Initialized to 0 for *s* and ∞ for all other vertices.

• **Previous Pointer Map (prev[]):** Stores the preceding vertex along the shortest path discovered so far. Initialized to null.

• **Unvisited Set (Q):** Contains all vertices that have not yet been finalized. Initialized to *V*.

The Relaxation Formula:

```
When examining an outgoing edge from vertex u to neighbor v with weight w(u, v):
If dist[u] + w(u, v) < dist[v] ■ dist[v] = dist[u] + w(u, v), prev[v] = u
```

Why Non-Negative Weights are Required:

Dijkstra operates on a *greedy choice property*: once a vertex *u* with minimum tentative distance is extracted from the unvisited set *Q*, its distance dist[u] is declared final and never re-evaluated. If negative edge weights were present, a longer path with a negative edge could later yield a smaller total distance, violating the greedy assumption and producing incorrect results.

Time Complexity:

In our implementation (in dijkstra.js), minimum distance vertex selection is performed using a linear scan over the unvisited set *Q*. The time complexity is **O(V²)**, which is optimal and easy to explain for discrete mathematics educational projects.

5. Code Architecture & Implementation

The project follows a clean, modular React architecture:

File / Component	Function / Purpose
src/algorithms/dijkstra.js	Pure JS manual implementation of Dijkstra's algorithm & step tracer
src/components/GraphView.jsx	Interactive SVG canvas rendering draggable circular nodes & directed arrows
src/components/GraphControls.jsx	Form controls for adding vertices, directed edges, resetting, & clearing
src/components/ShortestPath.jsx	Source/Destination selector dropdowns & formatted result card
src/components/AdjacencyMatrix.jsx	Renders weighted $V \times V$ directed adjacency matrix table
src/components/AlgorithmTrace.jsx	Step-by-step distance relaxation table for viva demonstration
src/components/Theory.jsx	Course objective, graph definitions, and time complexity breakdown
src/components/Team.jsx	Team member details & registration numbers grid

6. Sample Working & Viva Demonstration

Default MAT2002 Demo Graph Verification:
• **Vertices:** A, B, C, D, E
• **Edges:** A → B (4), A → C (2), C → B (1), B → D (5), C → D (8), D → E (2), B → E (10)
• **Source:** A | **Destination:** E

Step	Current Vertex	Updated Distances (Relaxation)
1	**A**	B = 4, C = 2
2	**C**	B = 3 (via C: 2+1), D = 10 (via C: 2+8)
3	**B**	D = 8 (via B: 3+5), E = 13 (via B: 3+10)
4	**D**	E = 10 (via D: 8+2)
5	**E**	Destination reached. Search complete.

```
Result Output:
Shortest Path: A → C → B → D → E
```

```
Total Distance: 10
```

7. Top 20 Viva Questions & Answers

Q1: What is a graph in Discrete Mathematics?

A graph G = (V, E) is a discrete mathematical structure consisting of a non-empty set of vertices (V) and a set of edges (E) connecting pairs of vertices.

Q2: What is a directed graph (digraph)?

A directed graph is a graph where edges have an assigned direction. An edge (u, v) indicates traversal is permitted only from vertex u to vertex v.

Q3: What is a weighted graph?

A weighted graph is a graph in which a numerical value (weight or cost) is associated with each edge, representing metrics such as distance, travel time, or bandwidth.

Q4: What is the difference between a vertex and an edge?

A vertex (node) represents an individual entity or location in a graph, while an edge (link) represents a relationship or connection between two vertices.

Q5: What is a path in a graph?

A path is a sequence of vertices v1, v2, ..., vk such that there exists a directed edge from vi to vi+1 for all 1 <= i < k.

Q6: What is Dijkstra's algorithm and what problem does it solve?

Dijkstra's algorithm is a greedy single-source shortest path algorithm that finds the path with minimum total weight from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

Q7: What is 'distance relaxation' in Dijkstra's algorithm?

Relaxation is the step where a tentative distance to vertex v is updated if a shorter path is found via vertex u: if dist[u] + weight(u, v) < dist[v], then dist[v] is updated to dist[u] + weight(u, v).

Q8: Why does Dijkstra's algorithm require non-negative edge weights?

Dijkstra assumes that adding an edge to a path can never decrease its total weight (greedy choice property). Negative edge weights violate this assumption and can cause already finalized vertex distances to become invalid.

Q9: How does the algorithm reconstruct the final shortest path?

The algorithm maintains a previous pointer array prev[]. After execution, it traces backward from the destination vertex d using prev[d], prev[prev[d]], until reaching the source vertex s.

Q10: What is the time complexity of Dijkstra's algorithm in your project?

In our project, minimum distance selection is performed using a linear scan over the unvisited vertex array, giving a time complexity of O(V^2).

Q11: What happens if the destination vertex is unreachable from the source?

The distance to the destination remains Infinity (∞), no valid previous pointers link back to the source, and the app displays 'No path exists between the selected vertices.'

Q12: What is an Adjacency Matrix?

An adjacency matrix is a square matrix A of size V x V where element A[i][j] represents the weight of the directed edge from vertex i to vertex j.

Q13: How is the adjacency matrix represented in your project?

Self-loop entries A[i][i] are set to 0, existing directed edges A[i][j] contain their numeric weight, and non-existent edges are represented as Infinity (∞).

Q14: Why is an adjacency matrix useful for MAT2002 evaluation?

Adjacency matrix representation is a key topic in the MAT2002 course syllabus. It demonstrates how algebraic matrix operations can represent graph structures in computer memory.

Q15: How are vertices and edges stored in React frontend state?

Vertices are stored in a simple string array (['A', 'B', 'C']), and edges are stored as an array of objects ({ from: 'A', to: 'B', weight: 4 }).

Q16: Why did you implement Dijkstra's algorithm manually instead of using a library?

Implementing Dijkstra manually demonstrates a fundamental understanding of data structures, loops, distance relaxation, and path reconstruction required for academic viva assessment.

Q17: How does the application handle invalid user inputs?

It validates against duplicate vertex names, duplicate directed edges, negative weights, zero weights, empty fields, self-loops, and missing vertices using inline error messages.

Q18: What happens if the source and destination vertices are identical?

The distance is immediately set to 0, the path contains only that single vertex, and the app reports 'Source and destination are the same.'

Q19: How does the visual UI distinguish the shortest path?

Path edges are highlighted with thick emerald green strokes and custom arrowhead markers, path nodes receive green/blue rings, and non-path edges are rendered with subdued gray opacity.

Q20: What are real-world applications of Dijkstra's algorithm?

GPS navigation systems (Google Maps), network packet routing (OSPF protocol), flight connection planning, and supply chain logistics optimization.

8. Rapid Viva Revision Sheet

Key Terminology Cheat-Sheet:

Term	1-Line Viva Summary
Graph G=(V,E)	Set of vertices V connected by edges E
Directed Graph	Graph where edges have direction u → v
Weighted Graph	Graph where edges carry numeric costs w(u,v) > 0
Adjacency Matrix	V × V matrix storing edge weights, 0 for self, ∞ for no edge
Dijkstra Algorithm	Greedy algorithm for single-source shortest path with non-negative weights
Distance Relaxation	Updating dist[v] = dist[u] + w(u,v) if smaller than current dist[v]
Time Complexity	$O(V^2)$ for array-based implementation, O((V+E) log V) with Min-Heap

Dijkstra in 7 Steps (Memorize for Viva):

1. Set dist[source] = 0 and dist[v] = ∞ for all other vertices.
2. Set prev[v] = null for all vertices.
3. Initialize unvisited set Q = V.
4. Select unvisited vertex u ∈ Q with smallest dist[u].
5. For each neighbor v of u in Q: relax distance if dist[u] + w(u,v) < dist[v].
6. Remove u from Q.
7. Repeat until destination is reached or Q is empty. Reconstruct path backward using prev[].